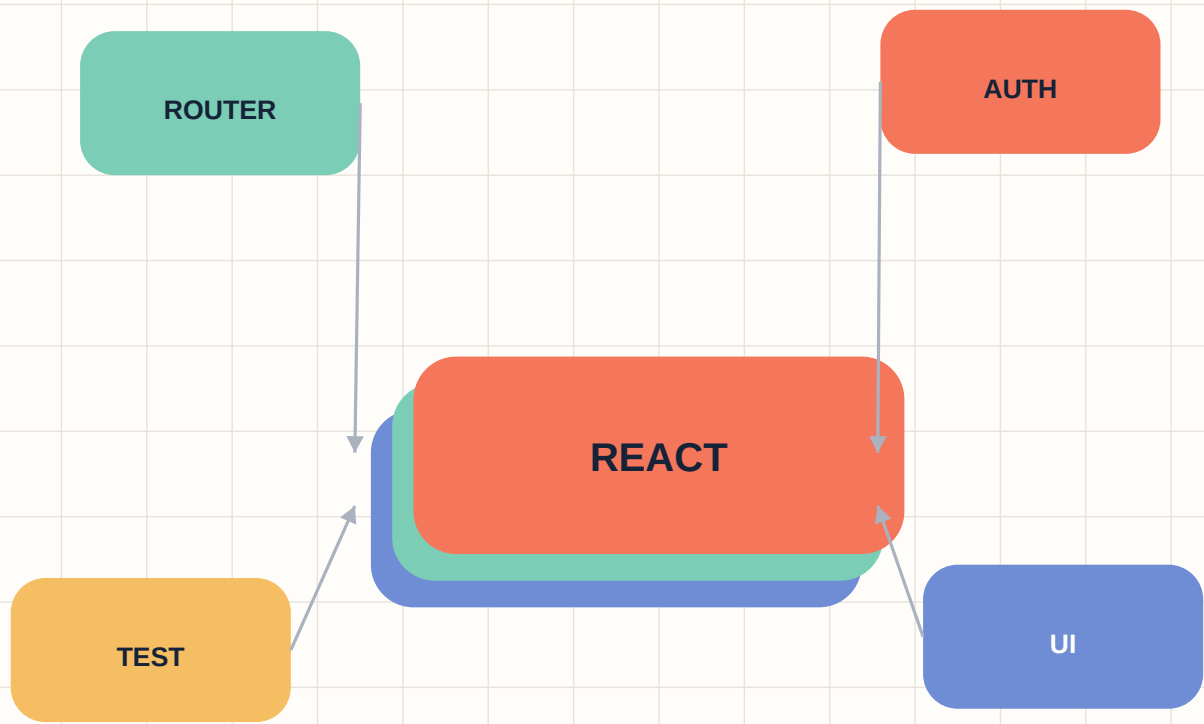




REACT

TYPESCRIPT

VITE

Start small. Add only what the app earns.

An option-driven React generator white paper

A small core. Explicit capabilities. Contradictions rejected before files exist.

Proof before posture.

Repository facts, not productivity theater.

28

PUBLIC OPTIONS

17 boolean and 11 enumerated choices.

47

TESTS PASSED

Current local repository test run; zero failed.

3

CONFLICT RULES

Each contradiction has actionable error coverage.

1

AUTO IMPLICATION

shadcn intentionally enables Tailwind.

What those numbers actually prove

- ✓ The option surface has an executable validator contract.
- ✓ Presence, absence, implications, and conflicts are tested.

They do not claim adoption, speed gains, or production performance.

A starter should not be a suitcase.

The useful change is not more code. It is visible choice.

CONVENTIONAL STARTER

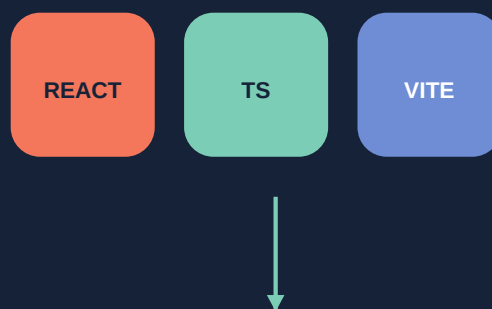
**Everything arrives
before the decision.**



Unused dependencies and structure
become cleanup work - or permanent
ambiguity.

OPTION-DRIVEN GENERATOR

**Core first.
Capabilities opt in.**



SELECTED ONLY

Dependencies / configuration /
source / tests

Disabled capabilities leave no
owned artifacts behind.

MINIMAL CONFIGURATION

2 runtime dependencies

React + ReactDOM

Verified by the dependency-plan
contract test.

Decide. Validate. Then write.

The filesystem is the last step, not the scratchpad.

01

PARSE

Read explicit --option=value flags.

02

VALIDATE

Normalize values and reject unknown input.

03

RESOLVE

Apply one documented implication; test conflicts.

04

COMPOSE

Plan selected dependencies, files, config, and tests.

05

WRITE

Create the application only after the plan is valid.

BOUNDARY AFTER GENERATION

Install, build, test, and deployment checks remain explicit user or CI work.

Generated is not certified.

A consistent artifact is the start of application ownership, not the end.

GENERATED CODE



PROJECT CONTEXT



READINESS



REVIEW

Check generated choices against the product and team.



DEPLOY

Validate runtime, data, observability, and failure behavior.



SECURE

Assess identity, secrets, permissions, and threat context.

ONE DELIBERATE NON-GOAL

No post-generation capability installer.

Existing applications require merging, source analysis, and a migration contract.

TAKEAWAY

Five rules worth keeping.

The template is code. The contract is the real product.

01

Keep the core boring.

A default earns its place only when nearly every app needs it.

02

Test absence.

A disabled capability must leave no owned artifact behind.

03

Fail before files.

Contradictions should stop generation with an actionable message.

04

Give ownership away.

Generated applications own their code and later architecture.

05

Verify the artifact.

Build and test generated projects, not only option functions.

SHORT VERSION

Pack a toolbox, not a furnished apartment.